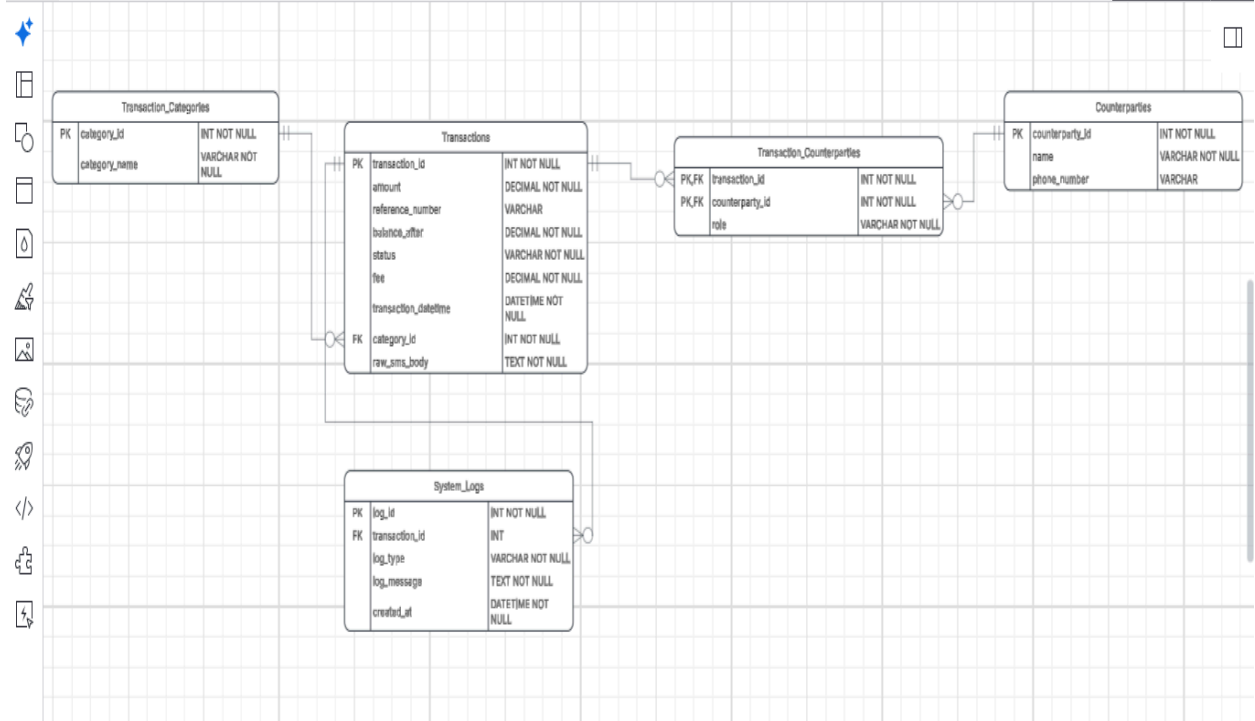


ERD Diagram



Design rationale and justification

MoMo SMS Data Processing System Database Design

The editable ERD is available in Lucidchart. A local image is available at [ERD Image](#).

ERD Design Decisions

Our schema was designed by working backwards from the actual MoMo SMS data rather than forward from assumptions about what a mobile money system should look like. The XML file contains 1,693 messages, and every structural decision we made came from patterns we found in those messages.

Why Counterparties instead of Users

We named the entity Counterparties instead of Users or Customers because the SMS data does not give us real user accounts. It gives us names and phone numbers extracted from message text, and those names include non-person entities like Airtime, Bundles and Packs, MTN Cash Power, and DIRECT PAYMENT LTD. A single Counterparties table with a name and nullable phone_number field handles all of them without special casing.

Why credit and debit is not its own field

Credit and debit is not stored as its own column because it is fully determined by the transaction category. A deposit is always a credit. A transfer out, an airtime purchase, a withdrawal, and a bill payment are always debits. Storing it separately would create a redundancy that could fall out of sync.

Why withdrawal, failed, and reversed are not separate tables

A withdrawal is just another category value in Transaction_Categories. Failed and reversed are outcomes captured by the status field on Transactions. Giving any of these their own table would duplicate every column that Transactions already has.

Why the many to many relationship exists

A single transaction can involve multiple parties. A withdrawal in our data involves the account holder, the agent, and the system. A transfer involves a sender and a receiver. The Transaction_Counterparties junction table resolves this with a composite primary key and a role field that records each party's function in the transaction.

Why System_Logs uses a nullable foreign key

System_Logs links to transaction_id with a nullable foreign key so it can record events that never produced a valid transaction, such as OTP messages or malformed records that failed during parsing.

Data dictionary with table and column descriptions

Data Dictionary: MoMo Analytics Database

This data dictionary describes the schema defined in `database_setup.sql` for the MOMO_Analytics database. It summarizes each table, its purpose, and the meaning of every column, along with integrity and usage notes.

Database Overview

The database stores MoMo transaction records, related counterparties (people, agents, merchants, and other parties), category metadata, and processing logs. The schema is designed to support transaction analysis, traceability, and auditability while enforcing data validation through constraints and triggers.

1. Transaction_Categories

Purpose: Stores the allowed business categories for each transaction, such as transfer, payment, deposit, withdrawal, or airtime purchase.

Column	Data Type	Nullability	Description	Constraints / Notes
category_id	INT	NO	Unique identifier for each transaction category.	Primary key; auto-incremented
category_name	VARCHAR(255)	NO	Name of the category, e.g., Transfer, Payment.	Must be unique and trimmed text cannot be empty

Business Rules:

- Each transaction must belong to one valid category.
- Category names are unique to avoid duplicate classifications.
- Deleting a category is blocked if it is still referenced by one or more transactions.

2. Counterparties

Purpose: Stores details of people, businesses, agents, or other parties involved in transactions.

Column	Data Type	Nullability	Description	Constraints / Notes
--------	-----------	-------------	-------------	---------------------

counterparty_id	INT	NO	Unique identifier for each counterparty.	Primary key; auto-incremented
name	VARCHAR(255)	NO	Name of the counterparty.	Must contain meaningful text
phone_number	VARCHAR(50)	YES	Phone number associated with the counterparty.	Optional because some counterparties may not have a known phone number; unique if provided

Business Rules:

- A counterparty may be an individual, merchant, or service provider.
- Phone numbers are optional and must be unique when entered.
- This table is linked to transactions through the junction table Transaction_Counterparties.

3. Transactions

Purpose: Stores the main financial transaction records created from SMS-based MoMo events.

Column	Data Type	Nullability	Description	Constraints / Notes
transaction_id	INT	NO	Unique identifier for each transaction.	Primary key; auto-incremented
amount	DECIMAL(18,2)	NO	Monetary value of the transaction.	Must be greater than 0
reference_number	VARCHAR(255)	YES	External or internal transaction reference.	Optional but unique when given
balance_after	DECIMAL(18,2)	NO	Account balance after the transaction.	Cannot be negative

status	VARCHAR(50)	NO	Current state of the transaction.	Default is completed; allowed values: completed, pending, failed, reversed
fee	DECIMAL(18,2)	NO	Service fee charged for the transaction.	Default 0.00; must not be negative
transaction_datetime	DATETIME	NO	Date and time the transaction occurred.	Future-dated values are rejected
category_id	INT	NO	Identifier of the transaction category.	Foreign key to Transaction_Categories
raw_sms_body	TEXT	NO	Original SMS message text used to generate the transaction record.	Used for traceability and future parsing audit

Business Rules:

- Each transaction must belong to a valid category.
- Amounts and fees must be realistic and non-negative.
- Transaction status must match supported lifecycle values.
- A trigger prevents future-dated transactions from being inserted.
- Every transaction creates an informational log entry automatically.

Relationship:

- Many transactions belong to one category.
- A transaction can involve one or more counterparties.

4. System_Logs

Purpose: Stores processing and audit logs for transactions or failures in the extraction and parsing workflow.

Column	Data Type	Nullability	Description	Constraints / Notes
log_id	INT	NO	Unique identifier for each log entry.	Primary key; auto-incremented
transaction_id	INT	YES	Transaction associated with the log.	Foreign key to Transactions; if transaction is deleted, log remains with NULL
log_type	VARCHAR(100)	NO	Type of log message.	Allowed values: info, warning, error, parse_success, parse_failure
log_message	TEXT	NO	Human-readable message describing the event.	Used for debugging and audit review
created_at	DATETIME	NO	Timestamp when the log was recorded.	Default is current timestamp

Business Rules:

- Logs can exist without a linked transaction if the event is unrelated to a valid record.
- Log types are controlled to maintain a consistent audit trail.
- Useful for monitoring ETL processing, parse errors, and important system events.

5. Transaction_Counterparties

Purpose: Represents the many-to-many relationship between transactions and counterparties.

Column	Data Type	Nullability	Description	Constraints / Notes
transaction_id	INT	NO	Identifier of the transaction.	Part of composite primary key; foreign key to Transactions
counterparty_id	INT	NO	Identifier of the counterparty involved.	Part of composite primary key; foreign key to Counterparties

role	VARCHAR(50)	NO	Role of the counterparty in the transaction.	Allowed values: sender, receiver
------	-------------	----	--	----------------------------------

Business Rules:

- One transaction can link to multiple counterparties.
- One counterparty can appear in multiple transactions.
- The composite primary key ensures that the same counterparty-role pairing is not duplicated in the same transaction.
- This resolves the many-to-many relationship between transactions and counterparties.

Relationship Summary

Transaction_Categories → Transactions

- One category can have many transactions.
- Transactions.category_id is a foreign key to Transaction_Categories.category_id.

Transactions → System_Logs

- One transaction can generate many logs.
- System_Logs.transaction_id references Transactions.transaction_id.

Transactions → Counterparties (via Transaction_Counterparties)

- Many transactions can involve many counterparties.
- The junction table resolves this relationship with a composite key and a role field.

Data Validation Notes

The schema applies validation to improve integrity and consistency:

- Category names must not be empty
- Counterparty names must not be empty
- Transaction amount must be positive
- Fee must not be negative
- Balance after transaction must not be negative
- Status must be one of the valid enumerated values
- Transaction date cannot be in the future
- Log type must be one of the defined system log types

Indexes

The database also defines indexes to improve query performance:

- idx_txn_datetime on Transactions(transaction_datetime)
- idx_txn_status on Transactions(status)
- idx_txn_category on Transactions(category_id)
- idx_txn_amount on Transactions(amount)
- idx_log_type on System_Logs(log_type)
- idx_log_created on System_Logs(created_at)
- idx_cp_name on Counterparties(name)

SQL Schema

```
CREATE DATABASE IF NOT EXISTS MOMO_Analytics;  
USE MOMO_Analytics;
```

```
CREATE TABLE Transaction_Categories (  
    category_id INT NOT NULL AUTO_INCREMENT,  
    category_name VARCHAR(255) NOT NULL,  
    PRIMARY KEY (category_id),  
  
    CONSTRAINT uq_category_name UNIQUE (category_name),  
  
    CONSTRAINT chk_category_name CHECK (CHAR_LENGTH(TRIM(category_name)) > 0)  
) ENGINE = InnoDB;
```

```
CREATE TABLE Counterparties (  
    counterparty_id INT NOT NULL AUTO_INCREMENT,  
    name VARCHAR(255) NOT NULL,  
    phone_number VARCHAR(50),  
    PRIMARY KEY (counterparty_id),  
    CONSTRAINT uq_phone_number UNIQUE (phone_number),  
    CONSTRAINT chk_counterparty_name CHECK (CHAR_LENGTH(TRIM(name)) > 0)  
) ENGINE = InnoDB;
```

```
CREATE TABLE Transactions (  
    transaction_id INT NOT NULL AUTO_INCREMENT,  
    amount DECIMAL(18,2) NOT NULL,  
    reference_number VARCHAR(255),  
    balance_after DECIMAL(18,2) NOT NULL,  
    status VARCHAR(50) NOT NULL DEFAULT 'completed',  
    fee DECIMAL(18,2) NOT NULL DEFAULT 0.00,  
    transaction_datetime DATETIME NOT NULL,  
    category_id INT NOT NULL,  
    raw_sms_body TEXT NOT NULL,  
    PRIMARY KEY (transaction_id),  
    CONSTRAINT uq_reference_number UNIQUE (reference_number),  
    CONSTRAINT chk_positive_amount CHECK (amount > 0),  
    CONSTRAINT chk_non_negative_fee CHECK (fee >= 0),  
    CONSTRAINT chk_non_negative_balance CHECK (balance_after >= 0),  
    CONSTRAINT chk_valid_status CHECK (status IN ('completed', 'pending', 'failed',  
'reversed')),  
    CONSTRAINT fk_transactions_category  
        FOREIGN KEY (category_id)  
        REFERENCES Transaction_Categories(category_id)  
        ON UPDATE CASCADE  
        ON DELETE RESTRICT
```

```
) ENGINE = InnoDB;
```

```
CREATE TABLE System_Logs (  
    log_id INT NOT NULL AUTO_INCREMENT,  
    transaction_id INT,  
    log_type VARCHAR(100) NOT NULL,  
    log_message TEXT NOT NULL,  
    created_at DATETIME NOT NULL DEFAULT NOW(),  
    PRIMARY KEY (log_id),  
    CONSTRAINT chk_valid_log_type CHECK (log_type IN ('info', 'warning', 'error',  
'parse_success', 'parse_failure'))),
```

```
    CONSTRAINT fk_logs_transaction  
        FOREIGN KEY (transaction_id)  
        REFERENCES Transactions(transaction_id)  
        ON UPDATE CASCADE  
        ON DELETE SET NULL
```

```
) ENGINE = InnoDB;
```

```
CREATE TABLE Transaction_Counterparties (  
    transaction_id INT NOT NULL,  
    counterparty_id INT NOT NULL,  
    role VARCHAR(50) NOT NULL,  
    PRIMARY KEY (transaction_id, counterparty_id),  
    CONSTRAINT chk_valid_role CHECK (role IN ('sender', 'receiver'))),
```

```
    CONSTRAINT fk_tc_transaction  
        FOREIGN KEY (transaction_id)  
        REFERENCES Transactions(transaction_id)  
        ON UPDATE CASCADE  
        ON DELETE CASCADE,  
    CONSTRAINT fk_tc_counterparty  
        FOREIGN KEY (counterparty_id)  
        REFERENCES Counterparties(counterparty_id)  
        ON UPDATE CASCADE  
        ON DELETE CASCADE
```

```
) ENGINE = InnoDB;
```

```
CREATE INDEX idx_txn_datetime ON Transactions(transaction_datetime);  
CREATE INDEX idx_txn_status ON Transactions(status);  
CREATE INDEX idx_txn_category ON Transactions(category_id);  
CREATE INDEX idx_txn_amount ON Transactions(amount);  
CREATE INDEX idx_log_type ON System_Logs(log_type);  
CREATE INDEX idx_log_created ON System_Logs(created_at);  
CREATE INDEX idx_cp_name ON Counterparties(name);
```

```

-- Block future-dated transactions (NOW() is not allowed in CHECK constraints)
DELIMITER //
CREATE TRIGGER trg_block_future_date
BEFORE INSERT ON Transactions
FOR EACH ROW
BEGIN
    IF NEW.transaction_datetime > NOW() THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Cannot insert a transaction with a future date.';
    END IF;
END //
DELIMITER ;

DELIMITER //
CREATE TRIGGER trg_log_new_transaction
AFTER INSERT ON Transactions
FOR EACH ROW
BEGIN
    INSERT INTO System_Logs (transaction_id, log_type, log_message, created_at)
    VALUES (NEW.transaction_id, 'info',
            CONCAT('Transaction ', NEW.transaction_id, ' inserted. Amount: ', NEW.amount, ' RWF.
Ref: ', IFNULL(NEW.reference_number, 'N/A')),
            NOW());
END //
DELIMITER ;
DELIMITER //
CREATE TRIGGER trg_prevent_category_delete
BEFORE DELETE ON Transaction_Categories
FOR EACH ROW
BEGIN
    DECLARE txn_count INT;
    SELECT COUNT(*) INTO txn_count
    FROM Transactions WHERE category_id = OLD.category_id;
    IF txn_count > 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Cannot delete category: transactions still reference it.';
    END IF;
END //
DELIMITER ;

INSERT INTO Transaction_Categories (category_name) VALUES
    ('Transfer'),
    ('Payment'),
    ('Deposit'),

```

('Withdrawal'),
('Airtime Purchase');

INSERT INTO Counterparties (name, phone_number) VALUES

('Jean Mugabo', '+250788000001'),
('MTN MoMo Agent', '+250788000002'),
('Kigali Water', '+250788000003'),
('Diane Uwase', '+250788000004'),
('Gasabo Agent', '+250788000005'),
('Airtime', NULL);

INSERT INTO Transactions (amount, reference_number, balance_after, status, fee,
transaction_datetime, category_id, raw_sms_body) VALUES

(5000.00, 'TXN20260901001', 45000.00, 'completed', 50.00, '2026-09-01 10:30:00', 1,
'You have sent 5,000 RWF to Jean Mugabo (250788000001). Fee: 50 RWF. Balance: 45,000
RWF. Ref: TXN20260901001'),
(20000.00, 'TXN20260902001', 65000.00, 'completed', 0.00, '2026-09-02 14:15:00', 3,
'You have received 20,000 RWF from MTN MoMo Agent (250788000002). Balance: 65,000
RWF. Ref: TXN20260902001'),
(3500.00, 'TXN20260903001', 61500.00, 'completed', 0.00, '2026-09-03 09:00:00', 2,
'Payment of 3,500 RWF to Kigali Water (250788000003) successful. Balance: 61,500 RWF.
Ref: TXN20260903001'),
(10000.00, 'TXN20260904001', 51500.00, 'completed', 100.00, '2026-09-04 16:45:00', 4,
'Withdrawal of 10,000 RWF at agent. Fee: 100 RWF. Balance: 51,500 RWF. Ref:
TXN20260904001'),
(500.00, 'TXN20260905001', 51000.00, 'completed', 0.00, '2026-09-05 08:20:00', 5,
'Airtime purchase of 500 RWF successful. Balance: 51,000 RWF. Ref: TXN20260905001'),
(15000.00, 'TXN20260906001', 36000.00, 'completed', 75.00, '2026-09-06 11:00:00', 1,
'You have sent 15,000 RWF to Diane Uwase (250788000004). Fee: 75 RWF. Balance:
36,000 RWF. Ref: TXN20260906001');

INSERT INTO Transaction_Counterparties (transaction_id, counterparty_id, role) VALUES

(1, 1, 'receiver'),
(2, 2, 'sender'),
(3, 3, 'receiver'),
(4, 2, 'receiver'),
(5, 6, 'receiver'),
(6, 4, 'receiver');

-- SAMPLE QUERIES

-- Q1: Total spending grouped by category

SELECT tc.category_name,
COUNT(*) AS txn_count,

```
SUM(t.amount) AS total_amount
FROM Transactions t
JOIN Transaction_Categories tc ON t.category_id = tc.category_id
GROUP BY tc.category_name
ORDER BY total_amount DESC;
```

```
-- Q2: All transactions with their counterparties
SELECT t.transaction_id, t.amount, t.status,
       t.transaction_datetime,
       c.name AS counterparty, tcp.role
FROM Transactions t
LEFT JOIN Transaction_Counterparties tcp ON t.transaction_id = tcp.transaction_id
LEFT JOIN Counterparties c ON tcp.counterparty_id = c.counterparty_id
ORDER BY t.transaction_datetime DESC;
```

```
-- Q3: Monthly summary
SELECT DATE_FORMAT(transaction_datetime, '%Y-%m') AS month,
       COUNT(*) AS total_transactions,
       SUM(amount) AS total_volume,
       SUM(fee) AS total_fees
FROM Transactions
GROUP BY month
ORDER BY month;
```

```
-- Q4: Failed or pending transactions
SELECT transaction_id, amount, status,
       reference_number, transaction_datetime
FROM Transactions
WHERE status IN ('failed', 'pending')
ORDER BY transaction_datetime DESC;
```

```
-- Q5: Recent system errors
SELECT log_id, transaction_id, log_type,
       log_message, created_at
FROM System_Logs
WHERE log_type IN ('error', 'parse_failure')
ORDER BY created_at DESC
LIMIT 20;
```

```
-- Q6: Top counterparties by transaction volume
SELECT c.name, c.phone_number,
       COUNT(*) AS txn_count,
       SUM(t.amount) AS total_volume
FROM Counterparties c
```

```
JOIN Transaction_Counterparties tcp ON c.counterparty_id = tcp.counterparty_id  
JOIN Transactions t ON tcp.transaction_id = t.transaction_id  
GROUP BY c.counterparty_id  
ORDER BY total_volume DESC;
```

Sample Queries

Q1, Total spending grouped by category

This joins Transactions to Transaction_Categories through category_id, then groups every row by category_name, counting how many transactions fall in each category and summing their amounts. Sorted highest spend first. This answers "where is the money actually going," transfers versus payments versus deposits and so on.

Q2, All transactions with their counterparties

This starts from Transactions and LEFT JOINs through Transaction_Counterparties into Counterparties to pull in the name and role for each transaction. The LEFT JOIN matters here specifically because of your zero to many relationship, a transaction like a bank deposit that has no counterparty still shows up in the results, just with NULL in the counterparty and role columns instead of disappearing entirely. Sorted most recent first.

Q3, Monthly summary

This uses DATE_FORMAT to collapse every transaction_datetime down to just its year and month, then groups by that, counting transactions and summing both amount and fee per month. This gives you a trend view, how transaction volume and total fees moved month over month, rather than looking at individual records.

Q4, Failed or pending transactions

A straightforward filter, pulls out every transaction where status is failed or pending, showing the amount, reference number, and date. This is the query you'd run to check what didn't go through cleanly, most recent first.

Q5, Recent system errors

Filters System_Logs down to just entries where log_type is error or parse_failure, most recent first, capped at 20 rows with LIMIT. This is meant to be a quick health check query, what's gone wrong recently in your ETL processing, without scrolling through every log entry ever created.

Q6, Top counterparties by transaction volume

Starts from Counterparties instead of Transactions this time, joining through Transaction_Counterparties to Transactions, then grouping by each counterparty to count how many transactions they've been part of and sum the total amount. Sorted highest volume first. This is the query for "who is this account moving the most money with," whether that's a person, an agent, or a service like Airtime.

Unique rules

Primary keys and AUTO_INCREMENT

Every table has a PRIMARY KEY, category_id, counterparty_id, transaction_id, log_id, each set to AUTO_INCREMENT so MySQL assigns the next number automatically, you never have to track "what's the next id" yourself. A primary key's whole job is guaranteeing no two rows in that table can ever be mistaken for each other, and every foreign key elsewhere in your schema points back to one of these.

Transaction_Counterparties is the one exception worth understanding, its primary key isn't a single AUTO_INCREMENT column, it's a composite key, transaction_id plus counterparty_id together. That means the same transaction can link to counterparty 1 and counterparty 2 as two separate rows, since only the id fine, but it means the exact same pairing can never be inserted twice, MySQL rejects a duplicate (transaction_id, counterparty_id) combination outright. That's what makes it a proper junction table instead of just a list.

uq_category_name

Stops two categories from ever having the same name, so you can't accidentally end up with "Transfer" listed twice with different category_id values, which would silently split your Q1 totals across two rows instead of one.

uq_phone_number

Stops two different Counterparties rows from claiming the same phone number, since one number should always mean one real counterparty, not two separate identities for the same person. Worth remembering from earlier testing though, NULL phone numbers (like Airtime or Bundles and Packs) don't count as duplicates of each other under this rule, MySQL treats every NULL as distinct, so this only actually enforces uniqueness when a real number is present.

uq_reference_number

Stops the same MoMo transaction ID from ever being inserted twice as two separate rows, which matters a lot given your real data, if your ETL pipeline ever re-processes the same SMS message by mistake, this constraint is what stops it from becoming a duplicate transaction and inflating your totals. Same NULL exception applies here too, for the 9 orphan bundle confirmations with no reference number at all.